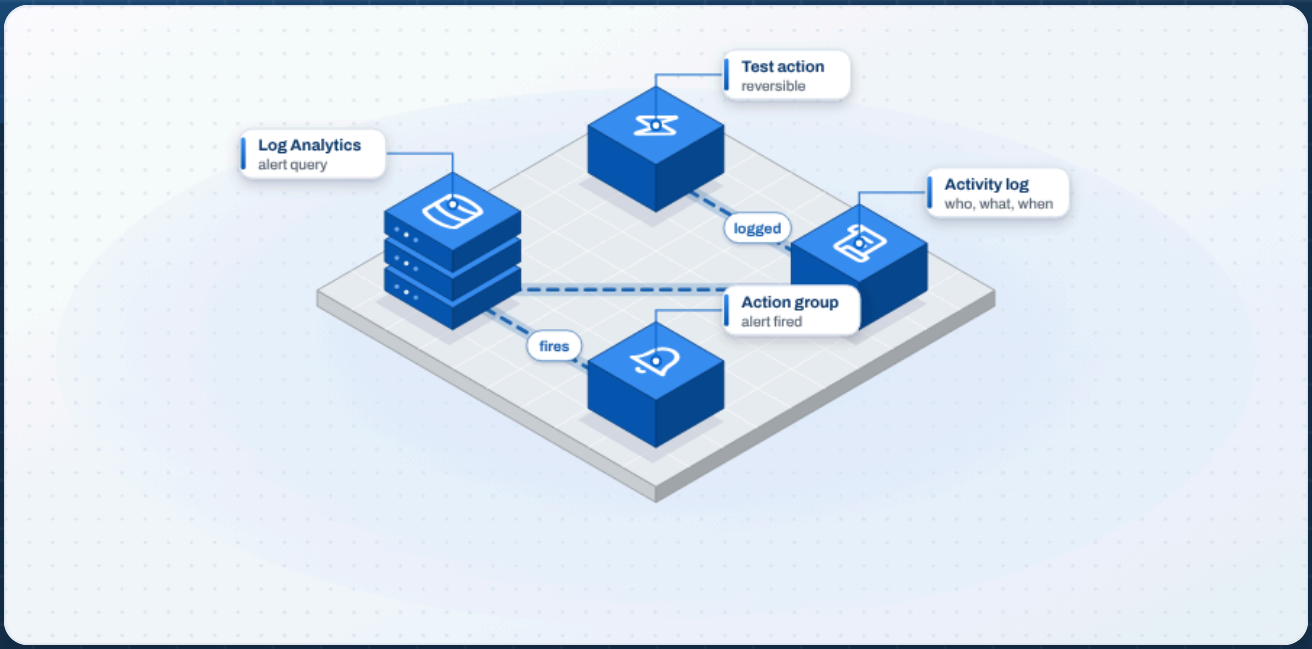
STANDARD OPERATING PROCEDURE · FIELD GUIDE SERIES

SOP-AZ-107

Validating Logging and Proving Every Alert Fires

Log source inventory, retention, alert rules for destructive actions, an audit trail of who, what and when, immutability, and a controlled test that fires every alert.

Security Operations | Log Analytics | Azure Monitor



DOCUMENT ID	SOP-AZ-107
VERSION	1.1
EFFECTIVE	October 2026
AUDIENCE	Detection engineers, SOC analysts, auditors
AUTHOR	Richmond Dieujuste, Principal Consultant

Rich Dieu IT Solutions LLC · richdieu.com

Sanitized reference edition · client identifiers removed

Contents

1. Concepts You Need First 2

2. Access, Prerequisites and Planning 4

2.1 Level of access required 4

2.2 Prerequisites checklist 4

3. Procedure in the Azure Portal 5

4. Template · Queries and Alert Test 8

5. Verification and Troubleshooting 10

5.1 Verify 10

5.2 Troubleshooting 10

6. Summary, Glossary and Review 11

6.1 Chapter summary 11

6.2 Glossary 11

6.3 Framework references 11

LEARNING OBJECTIVES

After working through this procedure you will be able to:

- Inventory log sources and retention and confirm the archive is immutable.
- Fire every alert rule with a harmless, reversible action and capture the timestamps.
- Show the who, what and when of each action in the workspace.

CHAPTER 1

Concepts You Need First

A logging design is only as good as its weakest assumption. A rule that has never fired may have a typo in its operation name, point at the wrong scope or email a mailbox nobody reads. The only proof is to make it fire.

KEY CONCEPT · TEST THE SMOKE ALARM

Nobody trusts a smoke alarm because it has a green light. You press the test button and listen. This SOP presses the button on every alert.

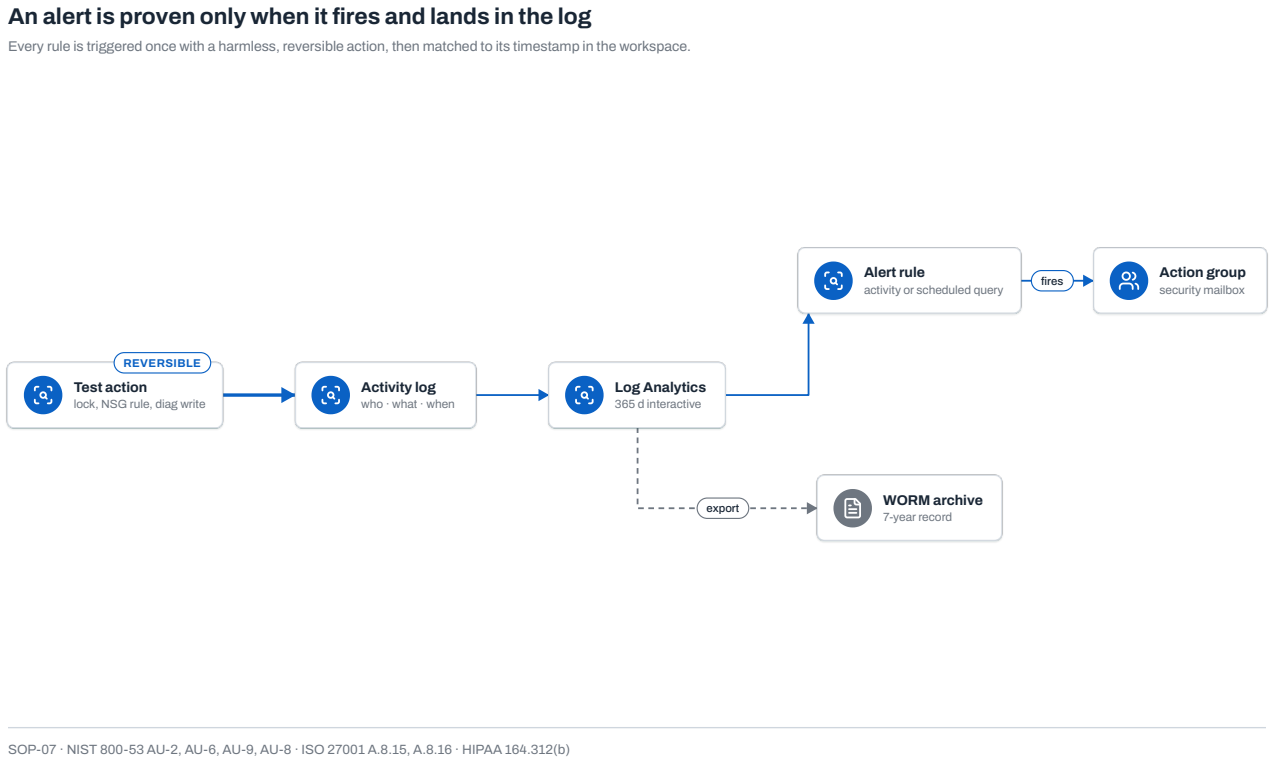


Figure 1. A reversible action is logged, reaches the workspace, trips the rule and reaches a person; the archive keeps the record.

Source	Captured by	Where it goes
Subscription activity log	Diagnostic setting	Workspace
Root management group activity log	Diagnostic setting set by API	Workspace
Key Vault, storage, vaults	Resource diagnostic settings (audit)	Workspace
VM syslog and performance	Azure Monitor Agent and a data collection rule	Workspace

Source	Captured by	Where it goes
Long-term record	Data export	WORM archive

CHAPTER 2

Access, Prerequisites and Planning

2.1 Level of access required

Task	Role	Notes
Read logs and rules	Log Analytics Reader, Monitoring Reader	No write
Fire alerts	Contributor on a dedicated test resource group	Only with written approval, only for the window
Receive alerts	Member of the action group mailbox	Or a copy of it

CAUTION · WRITTEN APPROVAL FOR THE TEST

The alert test makes real changes, even though each is reversed. Run it only inside a written test window and only in a dedicated test resource group.

2.2 Prerequisites checklist

- [ ] Test window approved in writing.
- [ ] Test resource group name agreed and tagged.
- [ ] Mailbox access to confirm alert emails.

CHAPTER 3

Procedure in the Azure Portal

NOTE · HOW THESE SCREENS WERE CAPTURED

Screens in this chapter were captured read-only from a live engagement tenant. Client, product and people names, tenant, subscription and object IDs, email addresses and public IPs are replaced (`org`, `app`, `Contoso`, `00000000-…`, `203.0.113.10`). Compliance figures and unrelated rows are blurred. Portal labels move over time; the setting names are what matter.

STEP 1 Inventory the log sources

1. For each resource open **Monitoring › Diagnostic settings**; confirm a setting sends audit logs to the central workspace.
2. Repeat for the subscription (Activity log export) and the root management group (template only).
3. Run KQL query 1 to see which tables are actually receiving rows.

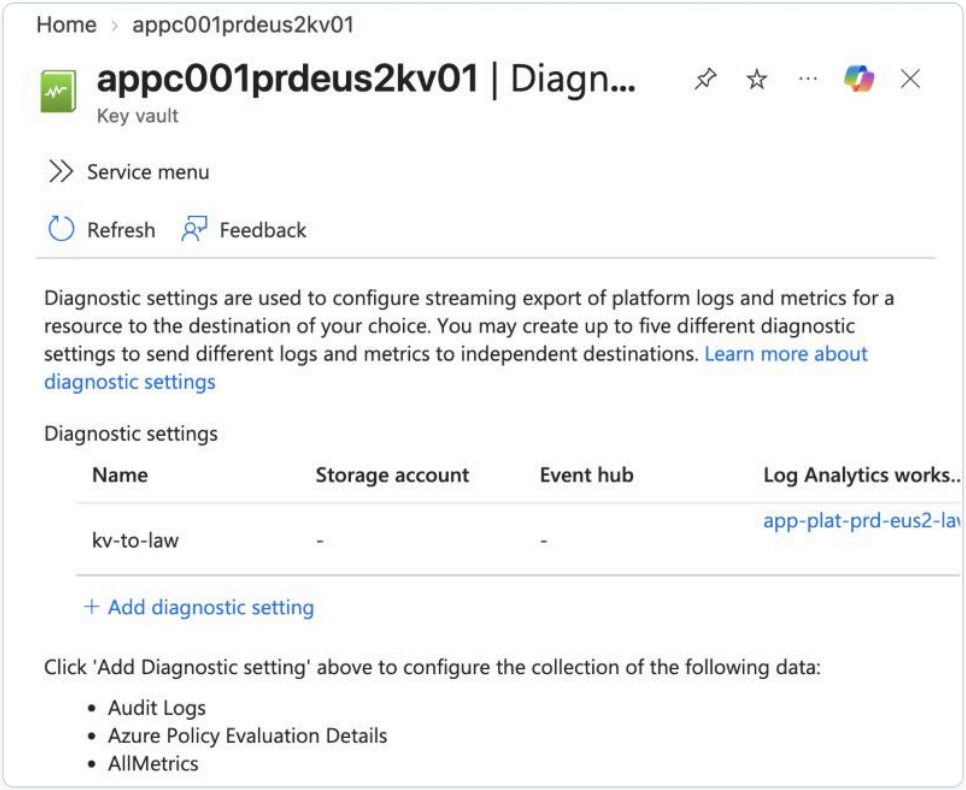


Figure 2. Key Vault diagnostic setting: audit logs and metrics to the central workspace.

STEP 2 Record retention

1. Open the workspace › **Settings › Tables** (or **Usage and estimated costs › Data retention**) and record interactive and total retention.
2. Open the archive storage account › **Containers**; record each container's time-based retention policy and lock state.

STEP 3 List the alert rules

1. **Monitor › Alerts › Alert rules.**
2. Expect one activity-log alert per destructive or privileged operation, plus scheduled-query rules for guardrail changes and the workload.

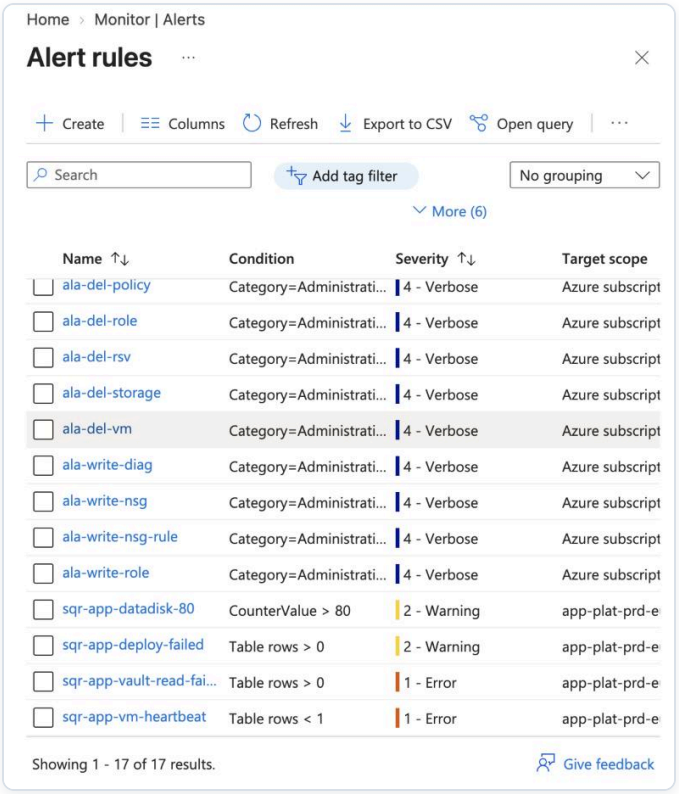


Figure 3. Alert rules: activity-log alerts for destructive and privileged operations, scheduled-query rules for the workload.

STEP 4 Fire every alert

1. With written approval run `SOP-07-alert-test.ps1` in the dedicated test resource group.

2. Each action is harmless and reversed at the end (create and delete a lock, add and remove an NSG rule).
3. The script writes a test log with the UTC time of each action.

STEP 5 Match fires to the log and close out

1. Wait 5 to 15 minutes. In **Monitor › Alerts** confirm each rule fired.
2. In the workspace run KQL query 2 to show who, what and when for each test action; put the alert time beside the action time.
3. Remove the test resource group, confirm its delete fired its own alert, and hash the test log into the manifest.

GOOD PRACTICE · MANAGED TIME

Record every time in UTC from the platform, not from a laptop clock. That is what makes the trail contemporaneous under ALCOA+ and 21 CFR Part 11.

CHAPTER 4

Template · Queries and Alert Test

SOP-07-ALERTS.KQL · LOG ANALYTICS · KQL TEMPLATES KQL

```kql
// SOP-07 templates: logging and detection validation (Log Analytics, KQL)

// 1. Log source inventory: which tables are receiving data, last 24 h
union withsource=Table *
| where TimeGenerated > ago(1d)
| summarize Rows = count(), Last = max(TimeGenerated) by Table
| order by Rows desc

// 2. Who / what / when for destructive or privileged control-plane actions
AzureActivity
| where TimeGenerated > ago(7d)
| where CategoryValue == "Administrative" and ActivityStatusValue in~ ("Success", "Succeeded")
| where OperationNameValue has_any ("/delete", "roleAssignments/write", "policyAssignments", "policyExemptions",
"locks/")
| project TimeGenerated, Caller, CallerIpAddress, OperationNameValue, _ResourceId, CorrelationId
| order by TimeGenerated desc

// 3. Guardrail-change alert query (scheduled query rule, severity 1, every 15 min)
AzureActivity
| where CategoryValue == "Administrative" and ActivityStatusValue in~ ("Success", "Succeeded")
| where OperationNameValue in~ ("Microsoft.Authorization/policyAssignments/write",
"Microsoft.Authorization/policyAssignments/delete",
                              "Microsoft.Authorization/policyExemptions/write",
"Microsoft.Authorization/policyExemptions/delete")
    or (OperationNameValue in~ ("Microsoft.Authorization/roleAssignments/write",
"Microsoft.Authorization/roleAssignments/delete")
        and tolower(_ResourceId) has "managementgroups")

// 4. Key Vault access log (every data-plane call), for the secrets evidence
AzureDiagnostics
| where ResourceProvider == "MICROSOFT.KEYVAULT" and TimeGenerated > ago(7d)
| summarize Calls = count() by OperationName, CallerIPAddress, ResultSignature

// 5. Alert test log: every fired alert with its timestamp (D9 evidence)
AzureActivity
| where TimeGenerated > ago(1d) and CategoryValue == "Alert"
| project Fired = TimeGenerated, OperationNameValue, Alert = tostring(parse_json(Properties).ruleName),
_ResourceId
```

SOP-07-ALERT-TEST.PS1 · ALERT-FIRE TEST · REVERSIBLE, WRITTEN APPROVAL ONLY POWERSHELL

```powershell
<#  SOP-07 template: fire each destructive-action alert with a harmless, reversible action in a test RG.
    Run only with the client's written OK. Every step is undone at the end.                              #>
param([string]$TestRg = '<product>-alerttest-prd-eus2-rg-01', [string]$Location = 'eastus2')
$log = [System.Collections.Generic.List[object]]::new()
function Mark($what) { $log.Add([pscustomobject]@{ Action = $what; UtcTime = (Get-
Date).ToUniversalTime().ToString('o'); By = (Get-AzContext).Account.Id }) }

New-AzResourceGroup -Name $TestRg -Location $Location -Tag @{ Owner='<owner>'; Client='<client>';
DataClassification='Internal' } | Out-Null
$nsg = New-AzNetworkSecurityGroup -Name nsg-alerttest -ResourceGroupName $TestRg -Location $Location;  Mark
'write-nsg'
$nsg | Add-AzNetworkSecurityRuleConfig -Name test-deny -Priority 4090 -Direction Inbound -Access Deny -Protocol *
`
```

```
        -SourceAddressPrefix * -SourcePortRange * -DestinationAddressPrefix * -DestinationPortRange * | Set-
AzNetworkSecurityGroup | Out-Null; Mark 'write-nsg-rule'
New-AzResourceLock -LockName test-lock -LockLevel CanNotDelete -ResourceGroupName $TestRg -Force | Out-Null
Remove-AzResourceLock -LockName test-lock -ResourceGroupName $TestRg -Force | Out-Null;            Mark 'del-
lock'
Remove-AzResourceGroup -Name $TestRg -Force | Out-Null;                                          Mark 'cleanup'

$log | Export-Csv "./alert-test-log-$(Get-Date -Format yyyyMMddTHHmmZ).csv" -NoTypeInformation
# Then: wait 5-15 min, open Monitor > Alerts, and match each fired alert to a row in this log (KQL query 5).
```

CHAPTER 5

Verification and Troubleshooting

5.1 Verify

- [ ] Every alert rule has a fired timestamp in the test log.
- [ ] Activity log rows show caller, operation and UTC time for each test action.
- [ ] Archive containers show a time-based retention policy; lock state recorded.
- [ ] Test resource group removed and its delete is in the log.

5.2 Troubleshooting

Symptom	Likely cause and fix
Alert never fires	Operation name typo, wrong scope (subscription vs management group) or the rule is disabled.
Alert fires but no email	Action group email not confirmed, or the mailbox filters it.
Guardrail change not in the workspace	Root management group diagnostic setting missing.

CHAPTER 6

Summary, Glossary and Review

6.1 Chapter summary

- Every source is inventoried and its retention recorded.
- Every alert is fired once and matched to its log row.
- Times come from the platform, in UTC; the archive is immutable.

Check your understanding

1. Why is a rule that has never fired treated as unproven?
2. Where do management group changes get logged?
3. What makes the test actions safe to run?

6.2 Glossary

Action group	Who an alert notifies and how.
Scheduled query rule	An alert defined by a KQL query on a schedule.
ALCOA+	Attributable, legible, contemporaneous, original, accurate, plus complete, consistent, enduring, available.

6.3 Framework references

Framework	Controls
NIST SP 800-53 R5	AU-2, AU-3, AU-6, AU-8, AU-9, AU-11, SI-4
ISO/IEC 27001:2022	A.8.15, A.8.16, A.8.17
HIPAA Security Rule	164.312(b)
21 CFR Part 11	§11.10(e)